Experiment #: EEPROM

Purpose:

Investigate the PIC16F883 data EEPROM; read and write stored data, verify retention across power cycles, measure operation timing, and apply reliabe nonvolatile storage in embedded systems.

Standards and Reference:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide and PIC-AS project setup
- PIC16F883 Datasheet
  - Section 10.1
  - Section 10.3.1
  - Section 17.5
- DLG7137 Display Datasheet
- Lab 2 - PIC I/O

Equipment and Materials:

- PIC16F883 circuit with 4MHz crystal and suitable power supply
- PICkit 3 or equivalent, MPLAB X, and pic-as
- DLG7137 single-character display
- Oscilloscope, Logic analyzer, frequency counter, and DMM as needed
- Lab Book, Breadboard, and various components as needed.

Part 1: EEPROM Persistence Proof of Life

Goal:

Select "A" or "B" in RAM, deliberately save it to data EEPROM address 0x00 and continuously display the stored EEPROM value. Measure read/write timing and verify retention after a power cycle.

Required Program Behavior:

- Buttons: Use two active-low buttons with internal weak pullups, the select button (RB0) and write button (RB1). Debounce both pressing and releasing each button in software.
- Normal Display: When the write button is released, continuosly read EEPROM address 0x00 in the main loop and display the stored "A", "B", or "?" for any other value.
- Choose a value: While the write button is held, display the pending RAM selection, initially "A". Each valid select button press toggles the selection between "A" and "B". Pressing the select button while the write button is released leaves the selection unchanged.
- Save a value: A valid Write Button Press enters selection mode. On it's debounce release, save the pending RAM value to EEPROM once, wait for the EEPROM to finish, and resume displaying the stored value.

Experiment 9: EEPROM

Required Program Behavior:

- Write disipline: An EEPROM write may start only after a complete, debounced write button press-and-release cycle. Do not write at startup, during normal main-loop reading, when the select button is pressed, or repeatedly while the write button is held. Contact bounce on either button must not produce an extra write.
- Timing Diagnostics: RA1 is High only during each EEPROM read, and RC2 is High only during the EEPROM write operation, including the wait for completion. Keep both low otherwise. Exclude display-update time from the read diagnostic and preserve the uninterrupted EEPROM write-unlock sequence.

Before Lab:

Prepare or Reference:

- EEPROM SFR maps and read/write procedure, including the protected write sequence, write-complete indication, and interrupt precautions.
- Circuit schematic and electrical-loading checks for both buttons, display, and diagnostics.
- Flowcharts for the main loop, select button handling, write button handling, software debouncing, EEPROM read/write, and display updates.
- Predicted display and diagnostic behavior for unrecognized EEPROM data, selection without saving, saving on write button release, contact bounce, and power cycling.
- EEPROM Durability: From Sections 17.5 and 10.3.1 of PIC16F883 Datasheet.

EEPROM is useful because it's nonvolatile and can store data after power has been removed, but the memory cells only have a limited amount of write cycles. Higher temps can reduce endurance and unnecessary writes can shorten the usable life of the memory. Due to this, programs should only write to EEPROM when necessary, such as when a user deliberately save a new value, rather than continuously writing during the main loop.

- Endurence Calculation: Suppose a programming error repeatedly writes the same EEPROM address, starting another write as soon as the previous one finishes. Estimate the time to consume the minimum specified byte-endurence budget using the typical and max write times for both temperature ranges. The calculation is:

$T_{budget} = N_{writes} \cdot t_{writes}$

Worked example (-40°C to +85°C, typical write time):

$T_{budget} = 100{,}000 \text{ writes} \cdot 5 \text{ ms/write}$

$T_{budget} = 500{,}000 \text{ ms} = 500{,}000 / 100\text{s}$

$\boxed{T_{budget} = 500\text{s} = 8\text{mins } 20\text{s}}$

A mistaken EEPROM write inside the main loop could use up the specified endurance budget in minutes and permanently damage that address. Verify that your code ~~writes~~ cannot write repeatedly before leaving it running. Do not perform a destructive endurence test.

Experiment 9: EEPROM

Before Lab:

Figure 9.1A: Write Code

```
BANKSEL EEDAT           ;
MOVF    EEDAT, W        ;EEDAT not changed
                        ;from previous write
BANKSEL EECON1          ;
BSF     EECON1, RD      ;YES, Read the
                        ;value written
BANKSEL EEDAT           ;
XORWF   EEDAT, W        ;
BTFSS   STATUS, Z       ;Is data the same
GOTO    WRITE_ERR       ;No, handle error
:                       ;Yes, continue
BCF     STATUS, RP1     ;Bank 0
```

Figure 9.1B: Write Verify Code

```
BANKSEL  EEADR          ;
MOVLW    DATA_EE_ADDR   ;
MOVWF    EEADR          ;Data Memory
                        ;Address to read
BANKSEL  EECON1         ;
BCF      EECON1, EEPGD  ;Point to DATA memory
BSF      EECON1, RD     ;EE Read
BANKSEL  EEDAT          ;
MOVF     EEDAT, W       ;W = EEDAT
BCF      STATUS, RP1    ;Bank 0
```

Figure 9.1C: Read Code

```
        BANKSEL  EEADR          ;
        MOVLW    DATA_EE_ADDR   ;
        MOVWF    EEADR          ;Data Memory Address to write
        MOVLW    DATA_EE_DATA   ;
        MOVWF    EEDAT          ;Data Memory Value to write
        BANKSEL  EECON1         ;
        BCF      EECON1, EEPGD  ;Point to DATA memory
        BSF      EECON1, WREN   ;Enable writes

        BCF      INTCON, GIE    ;Disable INTs.
        BTFSC    INTCON, GIE    ;SEE AN576
        GOTO     $-2
Required
Sequence
        MOVLW    55h            ;
        MOVWF    EECON2         ;Write 55h
        MOVLW    AAh            ;
        MOVWF    EECON2         ;Write AAh
        BSF      EECON1, WR     ;Set WR bit to begin write
        BSF      INTCON, GIE    ;Enable INTs.

        SLEEP                   ;Wait for interrupt to signal write complete
        BCF      EECON1, WREN   ;Disable writes
        BCF      STATUS, RP0    ;Bank 0
        BCF      STATUS, RP1
```

Figure 9.2: Circuit Schematic

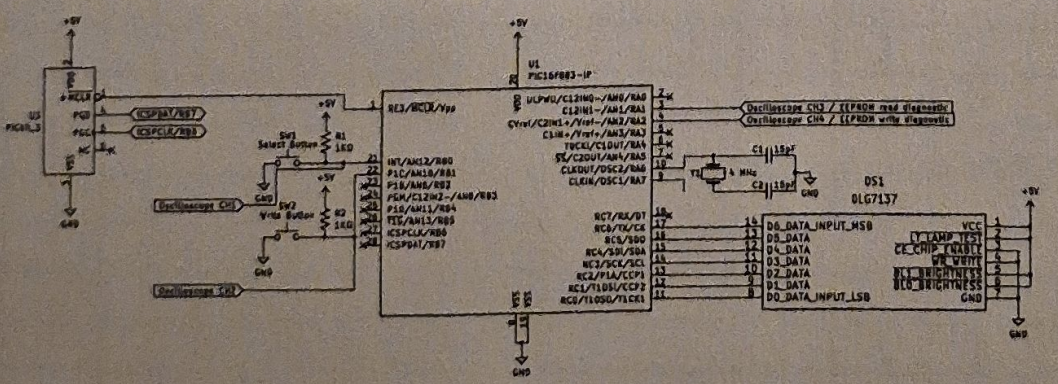


Figure 9.3: EEPROM endurance/ Refresh/ Retention Rate

		Data EEPROM Memory					
D120	ED	Byte Endurance	100K	1M	—	E/W	-40°C ≤ TA ≤ +85°C
D120A	ED	Byte Endurance	10K	100K	—	E/W	+85°C ≤ TA ≤ +125°C
D121	VDRW	VDD for Read/Write	VMIN	—	5.5	V	Using EECON1 to read/write VMIN = Minimum operating voltage
D122	TDEW	Erase/Write Cycle Time	—	5	6	ms	
D123	TRETD	Characteristic Retention	40	—	—	Year	Provided no other specifications are violated
D124	TREF	Number of Total Erase/Write Cycles before Refresh(4)	1M	10M	—	E/W	-40°C ≤ TA ≤ +85°C

Experiment 9: EEPROM

Before Lab:

Figure 9.1: EEPROM SFRs

REGISTER 10-1: EEDAT: EEPROM DATA REGISTER

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EEDAT7	EEDAT6	EEDAT5	EEDAT4	EEDAT3	EEDAT2	EEDAT1	EEDAT0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7-0 EEDAT<7:0>: Eight Least Significant Address bits to Write to or Read from data EEPROM or Read from program memory

REGISTER 10-2: EEADR: EEPROM ADDRESS REGISTER

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EEADR7	EEADR6	EEADR5	EEADR4	EEADR3	EEADR2	EEADR1	EEADR0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7-0 EEADR<7:0>: Eight Least Significant Address bits for EEPROM Read/Write Operation[1] or Read from program memory

REGISTER 10-3: EEDATH: EEPROM DATA HIGH BYTE REGISTER

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	EEDATH5	EEDATH4	EEDATH3	EEDATH2	EEDATH1	EEDATH0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7-6 Unimplemented: Read as '0'

bit 5-0 EEDATH<5:0>: Six Most Significant Data bits from program memory

REGISTER 10-4: EEADRH: EEPROM ADDRESS HIGH BYTE REGISTER

U-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	—	EEADRH4[1]	EEADRH3	EEADRH2	EEADRH1	EEADRH0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7-5 Unimplemented: Read as '0'

bit 4-0 EEADRH<4:0>: Specifies the four Most Significant Address bits or high bits for program memory reads

Note 1: PIC16F886/PIC16F887 only.

REGISTER 10-5: EECON1: EEPROM CONTROL REGISTER

R/W-x	U-0	U-0	U-0	R/W-x	R/W-0	R/S-0	R/S-0
EEPGD	—	—	—	WRERR	WREN	WR	RD
bit 7							bit 0

Legend:			
S = Bit can only be set			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 EEPGD: Program/Data EEPROM Select bit
1 = Accesses program memory
0 = Accesses data memory

bit 6-4 Unimplemented: Read as '0'

bit 3 WRERR: EEPROM Error Flag bit
1 = A write operation is prematurely terminated (any $\overline{MCLR}$ Reset, any WDT Reset during normal operation or BOR Reset)
0 = The write operation completed

bit 2 WREN: EEPROM Write Enable bit
1 = Allows write cycles
0 = Inhibits write to the data EEPROM

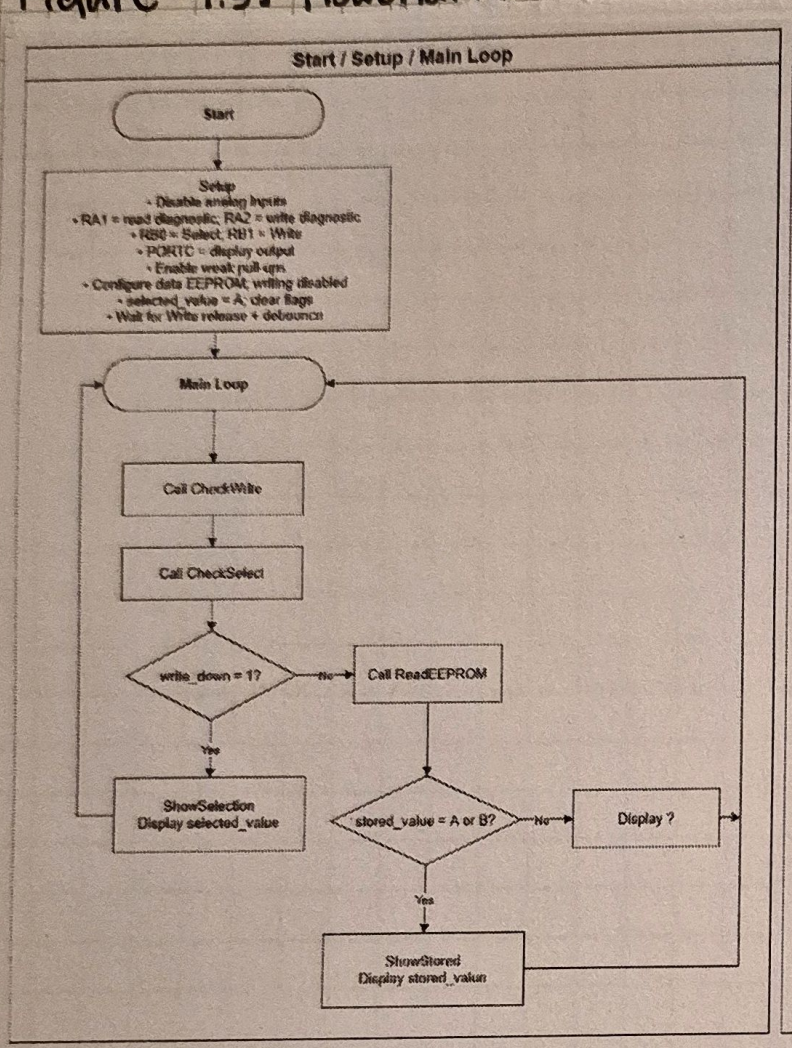
bit 1 WR: Write Control bit
1 = Initiates a write cycle (The bit is cleared by hardware once write is complete. The WR bit can only be set, not cleared, in software.)
0 = Write cycle to the data EEPROM is complete

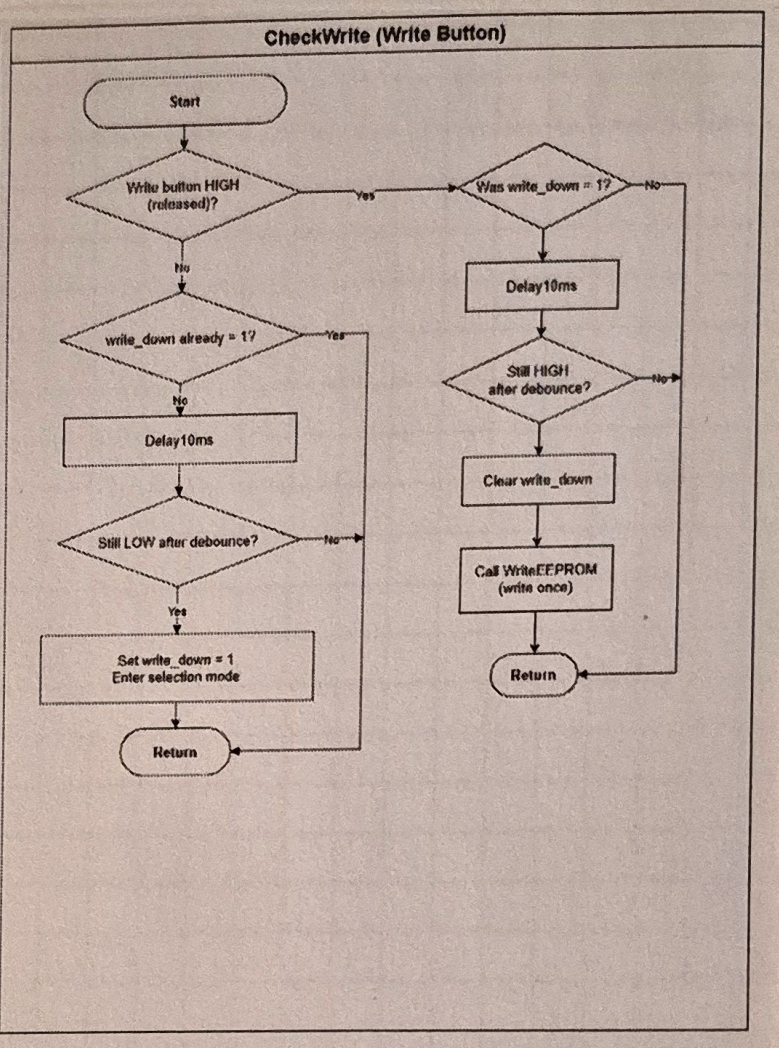
bit 0 RD: Read Control bit
1 = Initiates a memory read (the RD is cleared in hardware and can only be set, not cleared, in software.)
0 = Does not initiate a memory read

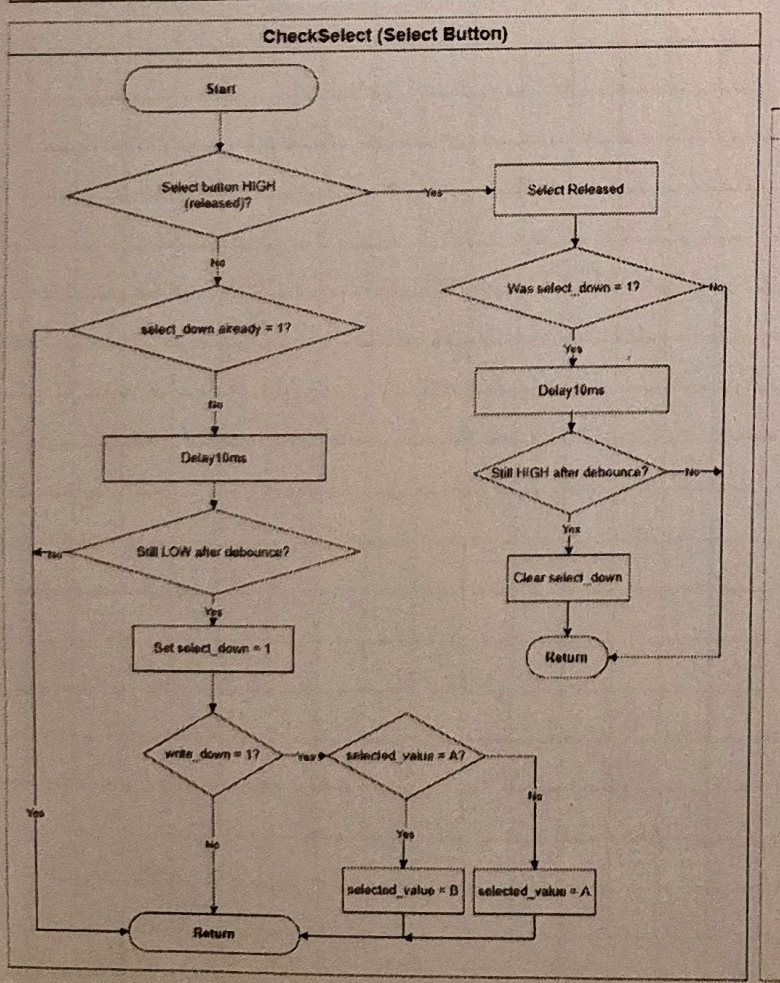
Experiment 9: EEPROM

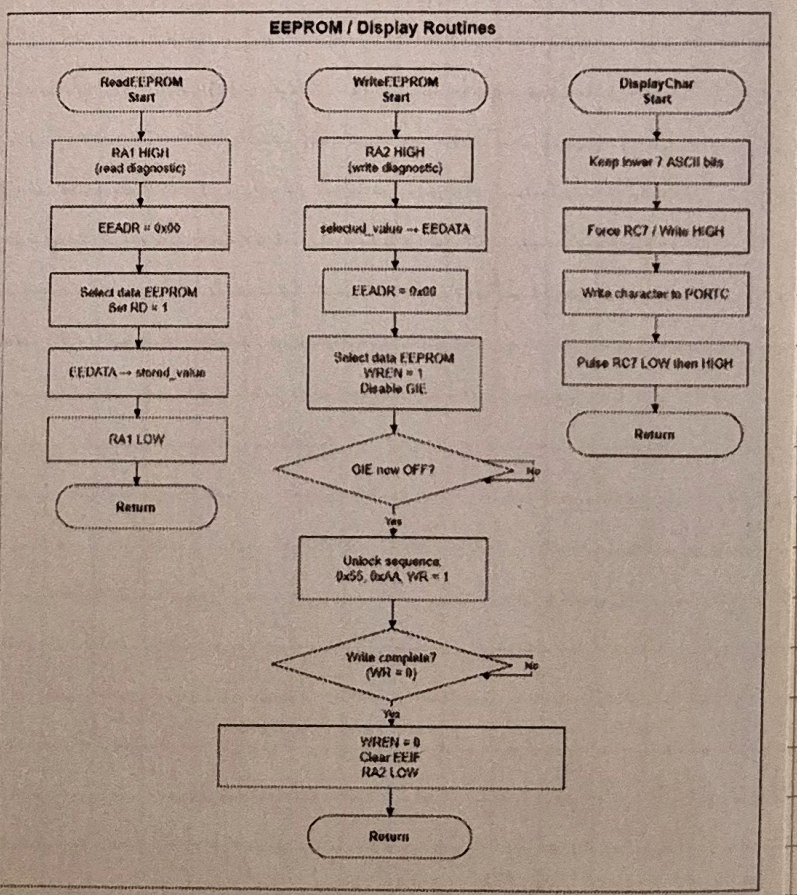
Before Lab:

Figure 9.5: Flowcharts









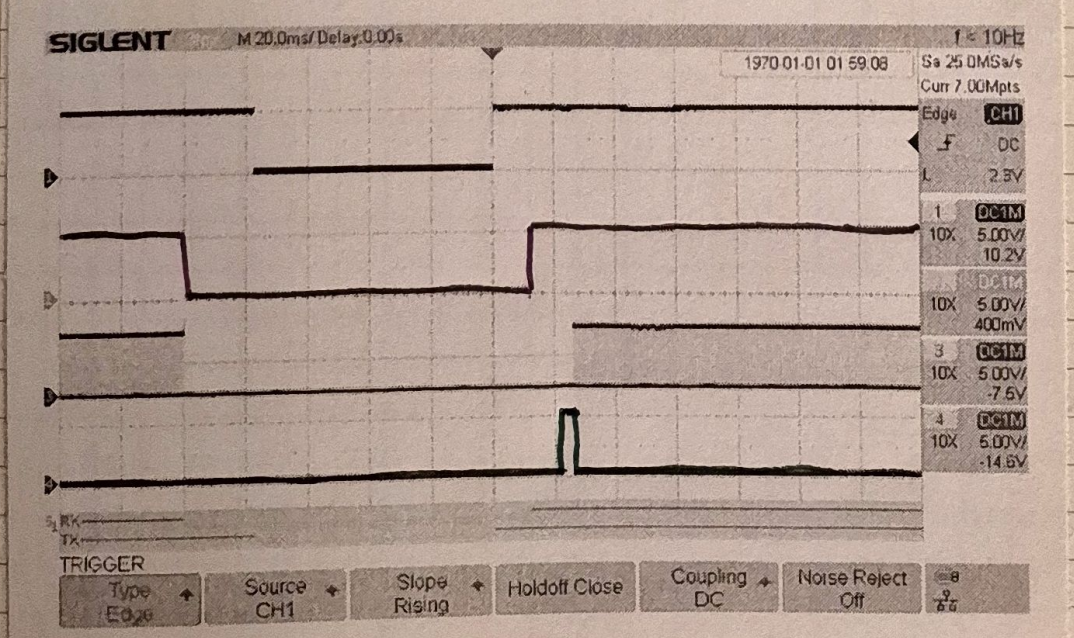
Experiment 9: EEPROM

In The Lab:

1.) Build the two-button and DLG7137 circuit with RA1 and RA2 available for oscilloscope measurements.
2.) Connect all four oscilloscope channels:
- Ch1 - Select button
- Ch2 - Write button
- Ch3 - EEPROM Read Diagnostic (RA1)
- Ch4 - EEPROM write Diagnostic (RA2)

3.) Verify continuous EEPROM reading with the write button released. Press and hold the write button to display the RAM selection, then use the select button to toggle between "A" and "B". Confirm that Ch4 stay low throughout selection.
4.) Trigger on both falling and rising edges of the select button (Ch1). Verify that it press and release do not produce an EEPROM write, whether or not the write button is held.
5.) Trigger on both falling and rising edges of the write button. Verify that the falling edge (press) starts selection mode without writing, and that only a valid debounced rising edge (release) leads to one Ch4 write pulse. Include sufficient post-trigger time for software debouncing and EEPROM programming.
6.) Record labeled four-channel captures showing these edge-trigger tests, including continuous Ch3 activity, contact bounce, and the Ch4 response. Use sufficient pre-trigger time to show selection before the write button release; use a seperate expanded acquisition to measure the short read pulses when needed.
7.) Measure and label both EEPROM read and write pulse widths in the scope captures. Compare write duration against the datasheet and account for diagnostic/software overhead.
8.) Test long holds, repeated select button presses, and physical press/releases bounce. Verify that each complete write button press-and-release produce exactly one write and the select button alone can't write.
9.) Save "A", remove all power, and restore it without touching either button or reprogramming. Verify the stored character appears, and repeat for "B".
10.) Inspect the EEPROM register and write-complete indication in the debugger. Make oscilloscope timing measurements during uninterrupted program execution.

Figure 9.6: Measured Read/Write Capture



TJR
10-20-26

Experiment 9: EEPROM

Part 2: Reusable EEPROM Read/Write Routines and Timed Selection

Goal:

Extend Part 1 with reusable EEPROM routines, a non-blocking write, and a seperate verification routine. Replace hold-to-select with a timed selection mode.

I/O Assignments:

Pin	Direction	Function
RA0	Output	Selection-mode LED, HIGH during selection
RA1	Output	Main-loop diagnostic pulse, once per iteration
RA2	Output	EEPROM error LED, HIGH when `EEPROM_ERROR` is set
RB0	Input	Select button, active LOW with internal pull-up
RB1	Input	Write button, active LOW with internal pull-up

Keep the DLG7137 connections on PORTC from Part1.

EEPROM routines:

GPR	Purpose
ee_address	EEPROM byte address for read or write
ee_data	Byte returned by a read or written during a write
ee_status	`EEPROM_READ`, `EEPROM_WRITE`, and `EEPROM_ERROR` status flags

Define the status flag bit positions with EQU. The EEPROM subroutines must use only these 3 RAM GPRs and work without modification in other programs. The calling application may use additional RAM GPRs. You may choose whether to use polling or the EEPROM write-complete interrupt to determine when a write is finished. If using the interrupt, the ISR should be called `EEPROMverifyWrite` once per complete write. You may use internal helper subroutines for EEPROM read/write, but call them only from the EEPROM routines and document the maximum stack depth. Use these routines for external calls:

- `EEPROMRead`: Load `ee_address` before the call. Set the `EEPROM_READ` status flag on entry, place the EEPROM byte in `ee_data`, and clear the `EEPROM_READ` status flag before returning.
- `EEPROMWrite`: Load `ee_address` and `ee_data` before the call. If `EEPROM_WRITE` or `EEIF` is set, return immediately without changing `ee_status`. Otherwise, compare the stored byte at `ee_address` with the requested value in `ee_data` without overwriting `ee_data`. If the values match, return without writing. Preserve any existing `EEPROM_ERROR` status flag. Otherwise, set the `EEPROM_WRITE` status flag, start the protected write sequence, and return without waiting for the write to finish. Keep the unlock sequence uninterrupted and follow the interrupt precautions from Part 1. Before returning, restore `GIE` to its previous state, enabling interrupts only if they were enabled on entry. You may use a bit in `ee_status` to save the previous `GIE` state; clear that bit after restoring `GIE`.
- `EEPROMVerifyWrite`: Call after the hardware write finishes. Compare the byte at `ee_address` with the expected value still in `ee_data`. Do not overwrite `ee_data` during the comparison. If they match, clear `ee_status`. If they differ, clear the `EEPROM_WRITE` status flag and set the `EEPROM_ERROR` status flag. Clear `EEIF` and `WREN` before returning.

Neither EEPROMWrite or EEPROMVerifyWrite should automatically retry a write on failure.

Experiment 9 : EEPROM

Button Service:

- Service the select and write buttons by polling or interrupts. Keep the Part 1 button debouncing and accept once per press.
- Do not start another EEPROM operation or change ee_address or ee_data while a write is awaiting verification.
- Ignore button events while either the EEPROM_Read or EEPROM_Write status flags are set. The EEPROM_ERROR status flag does not block the buttons.

Timed Selection:

- Normal operation: RA0 is off and display the character stored at EEPROM address 0x00.
- First Select Button (RB0) Press: Enter selection mode, select A, display A, turn RA0 on, and start an approximately 1sec timeout.
- Additional select button presses: Toggle the pending value between "A" and "B", update the display, and restart the 1sec timeout.
- Timeout: About 1sec after the last select button press, exit selection mode, turn off RA0, and resume the EEPROM stored-value display. A timeout doesn't start an EEPROM write.
- Write button (RB1): A debounced press before the timeout exit selection mode, stops the selection timer, turns RA0 off, and calls EEPROMWrite to save the pending character to address 0x00. Ignore write button presses outside selection mode.

Optional address-testing tip: In the application, define current_address with EQU as a RAM GPR and initialize it to 0x00. Use its value when loading ee_address for display reads and saved writes. Changing this one value makes it easier to test other EEPROM addresses.

Main-Loop Behavior:

Repeat these steps in MainLoop:

1.) Pulse RA1: set and immediately clear the main-loop diagnostic pin.
2.) Check write completion: If polling, check EEIF while the EEPROM_write status flag is set and call EEPROMVerifyWrite when the write finishes. If using the EEPROM write complete interrupt, the ISR calls writeVerify instead.
3.) Update RA2: Set the error LED to match the EEPROM_ERROR status flag on every iteration.
4.) Service timeout, then buttons: Check the selection timeout before accepting button presses. Poll the buttons or process button events supplied by interrupts.
5.) Update the DL67137 display: Show the pending character while in selection mode. Otherwise, read EEPROM address 0x00 and display "A", "B", or "?" for any other stored value. While the EEPROM_write status flag is set, keep displaying the last character without EEPROM.
6.) Repeat.

Experiment 9: EEPROM

Before Lab:

Document the 3 GPRs assignments, status flag bit positions, inputs and outputs of each EEPROM routine, and how GIE is saved and restored during the protected period. Prepare a state diagram for selection, timeout, writing, verification, and errors. Include the one-sec timing plan and an address/value test table. Document the RA0 and RA2 LED connection and loading, and the RA1 diagnostic pin. Explain why EEPROM reads do not consume write endurance and why checking for an unchanged value reduces wear.

Figure 9.7: Updated Schematic

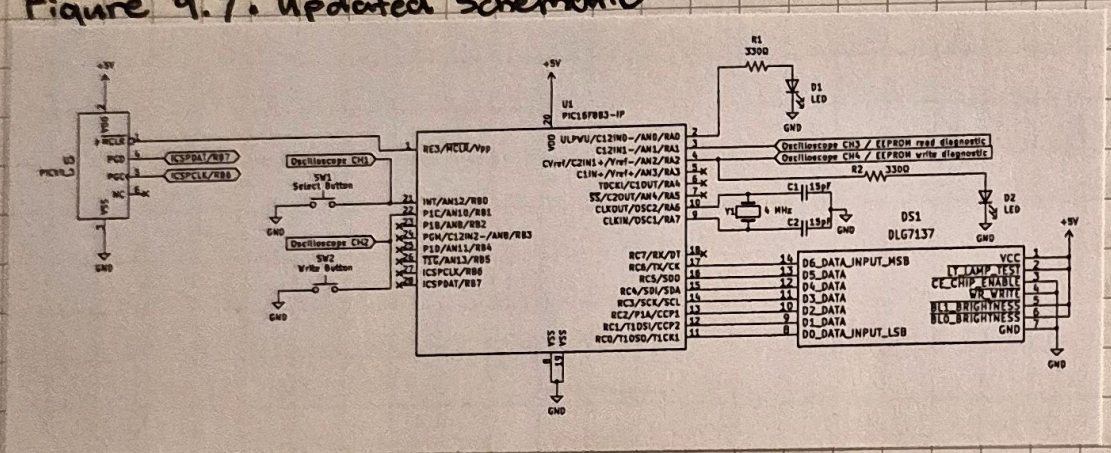
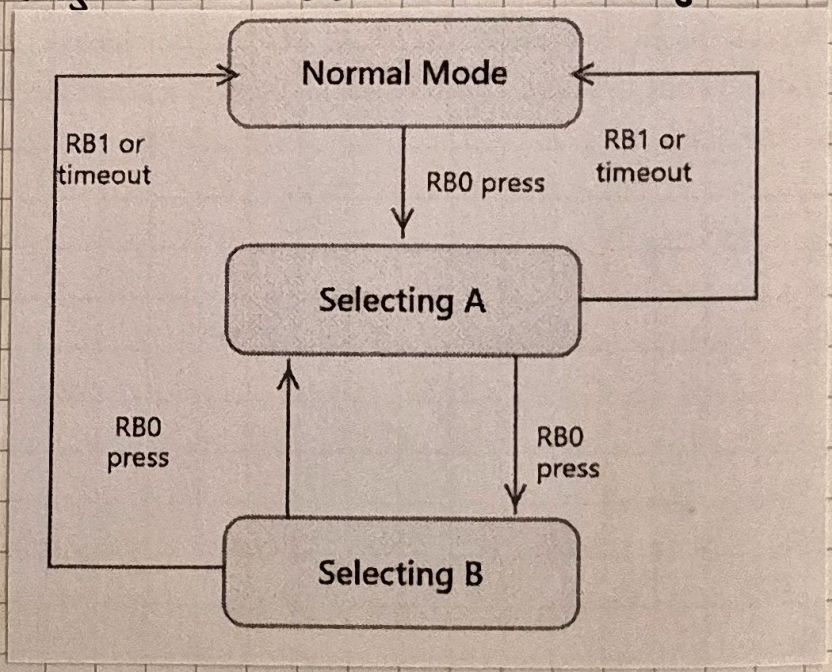


Figure 9.8: GPR Code

```
#include <xc.inc>
    ; General Purpose Registers
    selected_value EQU 0x20 ; Pending RAM selection: A or B
    stored_value   EQU 0x21 ; Value read from EEPROM
    write_down     EQU 0x22 ; 1 when Write button is held
    select_down    EQU 0x23 ; 1 when Select button is held
    Count5ms       EQU 0x24 ; Outer debounce delay counter
    Count50us      EQU 0x25 ; Inner debounce delay counter

    ; Reusable EEPROM registers (only these 3 GPRs used by EEPROM routines)
    ee_address     EQU 0x26  ; EEPROM address
    ee_data        EQU 0x27  ; Byte to read or write
    ee_status      EQU 0x28  ; EEPROM status bits

    ; Additional application registers
    selection_mode EQU 0x29  ; Bit 0 = selection mode active
    timeout_tick   EQU 0x2A  ; Counts 20 Timer1 overflows

    ; Constant (a number, NOT a register)
    TIMEOUT_OVERFLOWS EQU 15 ; 15 x 65.536 ms = about 1 second

    ; EEPROM status bit positions
    EEPROM_READ    EQU 0
    EEPROM_WRITE   EQU 1
    EEPROM_ERROR   EQU 2
    GIE_SAVED      EQU 3     ; Temporarily save GIE during unlock

    ; EEPROM address
    DATA_EE_ADDR   EQU 0x00  ; Store our character at address 0
```

Figure 9.9: Selection State Diagram

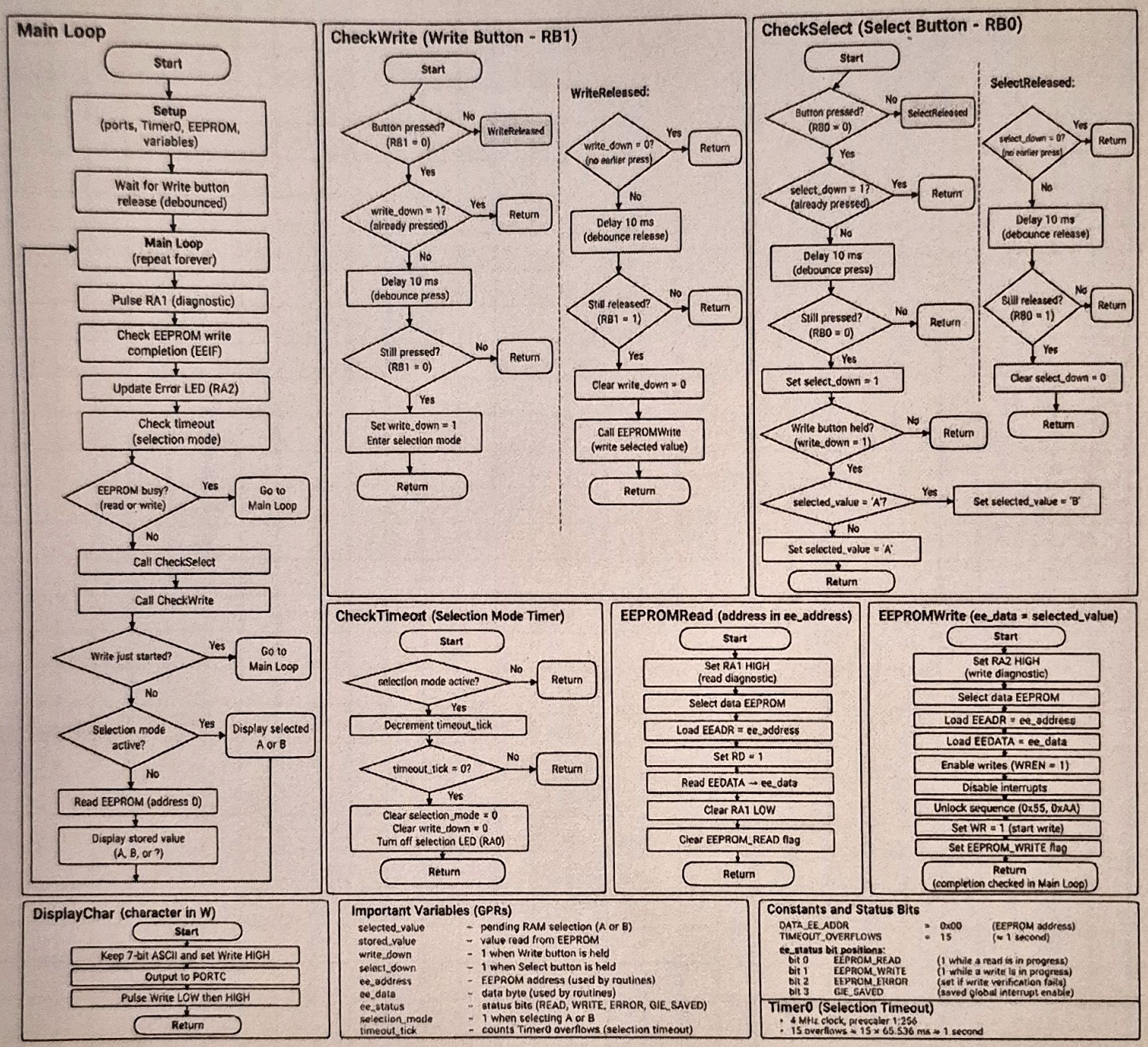


Part 2 was expanded my Part 1 EEPROM program to improve how the PIC16F883 selects, stores, and verify data. The program still uses RB0 to select between the characters "A" or "B" and RB1 to save the selected character to EEPROM, but now includes a Timer0 timeout that automatically exits selection mode after approximately 1 sec. Reusable read, write, and verification routines were added that prevent unnecessary writes, allowing the main program to continue running during EEPROM operation and error detection. The RA0 and RA2 outputs were both updated to display selection or errors. For the full code, plese refer to The github link below for Part 2:
https://github.com/hunterclezie-pixel/Lab-09---EEPROM.git

Experiment 9: EEPROM

Before Lab:

Figure 9.10: Flowcharts



Experiment 9: EEPROM

In The Lab:

1.) Refactor the code from part 1 into 3 named routines. Test several byte values at different EEPROM addresses and check them again after removing and restoring power.
2.) Check the EEPROM_Read status flag on entry to and from exit from EEPROMRead. Confirm the returned byte is in ee_data.
3.) Confirm EEPROM write returns with the EEPROM_write status flag set before hardware programming finishes.
4.) Handle EEIF and call EEPROMWriteverify once per completion write. Confirm that the ~~returned to~~ HIC verification clears ee_status. Test that buttons are ignored while the EEPROM_Read or EEPROM_write status flag is set.
5.) Test verification failiure by deliberately programming a different byte from the expected value retained in ee_data. Confirm that EEPROM_ERROR is set and the RA2 LED is on.
6.) Try saving the value already stored. Confirm no EEPROM write starts and any previous EEPROM_ERROR flags remain unchanged.
7.) Measure the selection timeout. Confirm that write is ignored outside selection mode but ~~exists~~ selection mode immediately when accepted.
8.) Capture RA1 during normal operation and an EEPROM write. Check that RA2 follows the EEPROM_ERROR status flag on every main-loop iteration.

Figure 9.11: EEPROM WriteVerify Error additions

```
EEPROMVerifyWrite:
    ; Read back the byte at ee_address
    CALL EEPROMFetch

    ; Compare with the expected ee_data (ee_data is not changed)
    BANKSEL ee_data
    MOVLW 0xFF          ; Testing Error
    XORWF ee_data,W
    BTFSC STATUS,2
    GOTO VerifySuccess
```

TJ12 10-22-25

By adding MOVLW 0xFF to the EEPROMVerifyWrite routine, it replaces the actual read-back value with 0xFF to simulate a verification error. When this value doesn't match the expected ee_data, the program sets the EEPROM_ERROR flag and the LED on RA2 turns on.

Experiment 9: EEPROM

Part 3: Four-digit Electronic Lock

Goal:

Extend Part 2 into a key-pad controlled electronic lock. Enter a four-digit PIN followed by "#" to activate a simulated solenoid output for 1sec. The Pin is read from data EEPROM.

Initial Pin setup:

Before running Part 3, manually program a four-digit PIN into four consecutive data EEPROM addresses, with one decimal digit (0-9) in each address. Choose the address and verify the stored digits. This is a one-time setup. PIN changes are not apart of this part and the pin must not appear anywhere in the program source.

Reuse from Previous Lab:

Use EEPROMRead from Part 2 without changing its interface. Pin entry and display handling belong to the application, not the EEPROM routines. Bring back the keypad scanner and DLG7137 display from Lab 2. A pin-entry routine can work like the ParseRx from the UART Lab: Recognise one key-press event at a time, collect digits in RAM, and track how many digits have been entered.

Keypad Behavior:

Use an active buzzer that produces its own tone when powered. Sound the buzzer for 100 to 250 ms on every debounce press of any of the 16 keys. This is button feedback only, it does not indicate whether the Pin is correct or incorrect.

Key	Action
0 – 9	Add one digit to the PIN entry in RAM
*	Cancel the entry and clear the collected digits
#	Submit the entry for comparison with the stored PIN
A – D	No PIN-entry action

Stored PIN and startup:

Use EEPROMRead to retreive the four stored PIN digits. On startup or reset, keep the solenoid

Experiment 9: EEPROM

Before Lab:

Prepare or reference the keypad scanner, display wiring, and debouncing from Lab 2, including electrical precautions or simultaneous key presses. Document the buzzer and simulated-solenoid output wiring, loading calculations, and reset-safe output behavior.

Figure 9.12: Keypad Lock Schematic

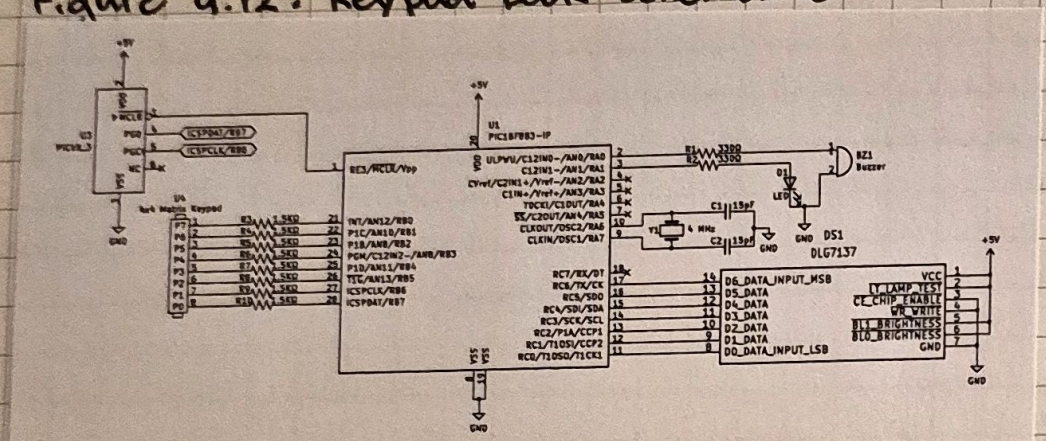
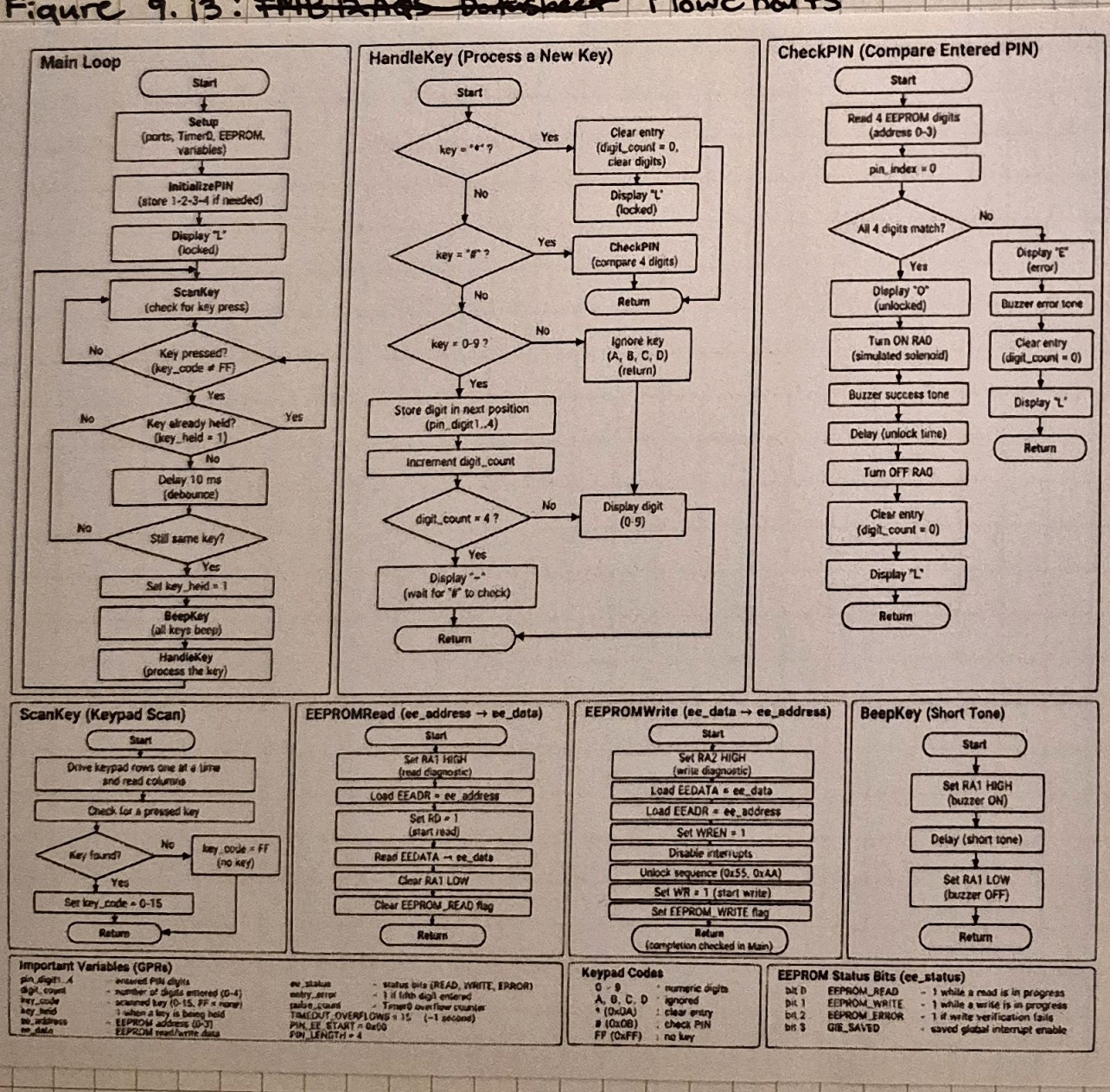


Figure 9.13: ~~FMB12A05 Datasheet~~ Flowcharts



Experiment 9: EEPROM

Before Lab:

Figure 9.14: TMB12A05 Datasheet

型 号 Type	TMB-12A01	TMB-12A03	TMB-12A05	TMB-12A12
额定电压(V) Rated Voltage	1.5	3	5	12
电压范围(V) Opcration Voltage	1.2~2.5	2.5~4	4~7	8~15
额定电流(mA) Rated Current	≤20	≤35	≤30	≤20
声压电平(dB) Sound Output	≥80	≥82	≥85	≥85
谐振频率(Hz) Resonant Freq	2300±500	2300±500	2300±500	2300±500
工作温度(℃) Operating Temp	-20~+60	-20~+60	-20~+60	-20~+60
储存温度(℃) Storage Temp	-20~+70	-20~+70	-30~+85	-30~+85
重 量(g) Weight	2	2	2	2

Based on observations, the current measured was 23mA and is out of specification. However, with the circuit using the TMB12A05 for only 150-250ms, allowing for the component to be used safely.

Figure 9.15: Initialize Pin code

```
; This is for the pin to initiallize, this can be deleted after flashing code
InitializePIN:
    BANKSEL ee_address

    ; First digit
    MOVLW 0x00
    MOVWF ee_address
    MOVLW 0x01
    MOVWF ee_data
    CALL WritePINByte

    ; Second digit
    MOVLW 0x01
    MOVWF ee_address
    MOVLW 0x02
    MOVWF ee_data
    CALL WritePINByte

    ; Third digit
    MOVLW 0x02
    MOVWF ee_address
    MOVLW 0x03
    MOVWF ee_data
    CALL WritePINByte

    ; Fourth digit
    MOVLW 0x03
    MOVWF ee_address
    MOVLW 0x04
    MOVWF ee_data
    CALL WritePINByte

    RETURN

WritePINByte:
    CALL EEPROMWrite

WaitPINWrite:
    BANKSEL EECON1
    BTFSC EECON1,1
    GOTO WaitPINWrite

    CALL EEPROMVerifyWrite
    RETURN
```

This section of code initializes the four-digit Pin (1234) by storing each digit in a seperate EEPROM address, from 0x00 to 0x03. The InitializePIN routine loads each address and digit before calling WritePinByte, which writes the value into EEPROM. The WaitPINWrite routine waits for the write operation to finish, and EEPROMVerifyWrite checks that the data was stored correctly. Since EEPROM retains it's data after power is removed, the initialization code can be removed after successfully programming. For the full code, refer to Part 3 of this Link:
https://github.com/hunterctezic-pixel/Lab-09---EEPROM

Experiment 9: EEPROM

In The Lab:

1.) Bring the keypad and display into the project and verify debounced key press produces one key event.
2.) Verify that the lock reads the PIN already stored in EEPROM.
3.) Enter the correct Pin and confirm that the solenoid output only activates after "#". Measure the 1s High pulse.
4.) Test an incorrect pin, "#" after fewer than 4 digits, a 5th digit, and "*" during entry. None may unlock.
5.) Verify that the buzzer works on all 16 keys.
6.) Power-cycle the PIC and confirm that the solenoid output stays inactive until the saved Pin is output again.

TSR
10-23-26

Experiment 9: EEPROM

Part 4: Authenticated PIN Change

Goal:

Extend the Part 3 lock so that the user can set a PIN on first use and change it with the keypad. Store the pin and its validity in EEPROM.

Pin storage and Startup:

Use five consecutive EEPROM addresses: 4 for the Pin digits (0-9) and one for a validity marker. Choose and document the addresses.
- Marker = 0xC4: The stored PIN is valid.
- Any other marker value: The PIN is invalid.

Before the first Part 4 run, use the programmer to set the marker to a invalid value such as 0xFF.

Service modes:

At startup, the stored PIN-valid maker determines whether the lock begins in invalid Pin mode or normal locked mode. The keypad must be serviced in ~~written~~ all modes and must not be blocked for more than 20ms, except during the 1sec solenoid activation and when a new PIN is written to EEPROM. The buzzer must sound for 100-250 ms on every debounced key press in all modes. Holding a key must produce only one key press; another press is accepted only after the key is released and pressed again. Each PIN entry accepts only digit 0-9, followed by "#" after exactly 4 digits. Used the DG7B7 to show whether the current or new PIN is being requested and whether saving succeeds or fails. Choose and document the display characters.

Invalid PIN mode:

The solenoid never activates in invalid PIN mode.
1.) Enter a new four-digit PIN followed by "#"
2.) Enter the same four-digits followed by "#"
Matching entries are saved as the new PIN. After a successful save, the lock enters normal locked mode. A mismatch, incomplete entry, unexpected key, or "*" clears the entry and restarts invalid PIN mode without saving.

Normal Locked Mode:

1.) Enter the current four-digit PIN followed by "#" to unlocked.
2.) To change the PIN, press "C" followed by "#"
A correct PIN activates the simulated solenoid for approximately 1 sec. The lock remains in normal locked mode. An incorrect or incomplete entry and restarts normal locked mode. If "C" is not the first key or is not followed by "#", return to normal locked mode.

Experiment 9: EEPROM

PIN Change mode:

The solenoid never activates in PIN change mode.
1.) Enter the current four-digit followed by "#"
2.) Enter a new four-digit PIN followed by "#"
3.) Enter the new Pin again followed by "#"
If the current PIN is correct and the two new entries match, save the new PIN and return to normal locked mode. An incorrect current PIN, mismatched new pins, incomplete entry, unexpected key, or "*" cancels the change and returns to normal locked mode with the old PIN unchanged. While saving a new PIN, the lock ignores keypad inputs and keeps the solenoid off. EEPROM write failures use the pin error behavior described below.

Reusable PIN-Matching Routines:

Consider two application subroutines. Both can use indirect addressing with FSR and INDF for pin digits held in RAM.

- `CurrentPINMatch`: Collect the keypad sequence `dddd#`, where each `d` is a digit from `0` to `9`. Consider four consecutive GPR locations with a pointer or index. Store each digit and advance the pointer. The fifth key press must be `#`; do not store it or advance the pointer. `*` or any unexpected key aborts. After `#`, use `EEPROMRead` to compare the four entered digits with the stored PIN. Report whether they match. Use this routine for normal unlocking and for authenticating a PIN change; the calling code decides whether to activate the solenoid.
- `NewPINMatch`: Collect the keypad sequence `dddd#dddd#`. Consider eight consecutive GPR locations, four for each new PIN entry. Store each digit from `0` to `9` and advance the pointer. The fifth and tenth key presses must be `#`; do not store them or advance the pointer. `*` or any unexpected key aborts. After the final `#`, compare the two groups of four digits in RAM and report whether they match. This routine does not write EEPROM.

Shared helper routines may be useful. Use RAM GPRs as needed to track key entry, state, pointer or index values, match flags, and other working data. Document the GPRs assignment and how each routine uses them in the lab book.

EEPROM Pin Write Sequence (PINWrite):

Create an application subroutine named PIN Write to manage the six EEPROM write steps below:

Step	EEPROM location	Value to write
1	PIN-valid marker	`0x00` (invalid)
2	PIN digit 1	First new digit
3	PIN digit 2	Second new digit
4	PIN digit 3	Third new digit
5	PIN digit 4	Fourth new digit
6	PIN-valid marker	`0xC4` (valid)

Experiment 9: EEPROM

Before Lab:

Update the Part 3 state diagram to show invalid Pin mode, normal locked mode, Pin change mode, PINWrite, success, and error handling. Include the currentPINMatch and NewPinMatch flows. Draw a flowchart for PinWrite showing all six EEPROM steps and the verification/error decisions. Document the five EEPROM addresses, PIN_Update, display characters, and the PORTA error indicator with its loading calculation.

Figure 9.16: Updated Keypad Schematic

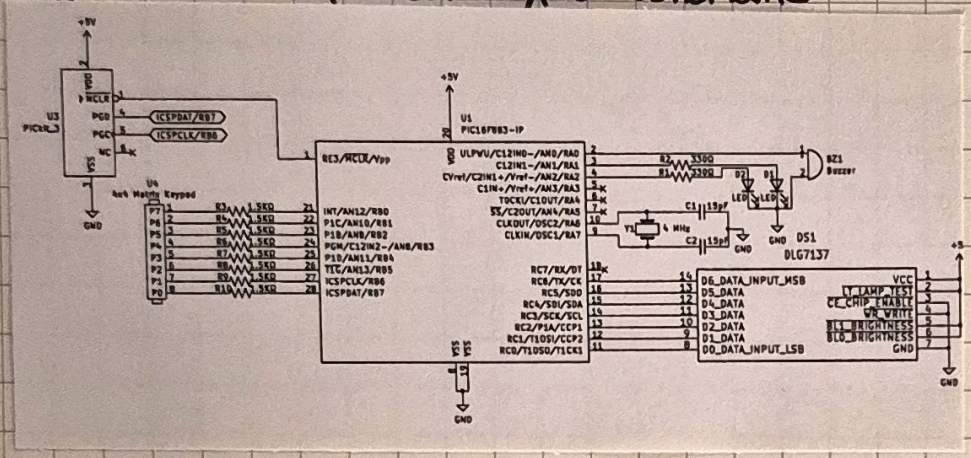


Figure 9.17: EEPROM Addresses

EEPROM data map

0x00	0x01	0x02	0x03	0x04
01	02	03	04	C4
Digit 1	Digit 2	Digit 3	Digit 4	Marker

Each PIN digit is stored as a numeric byte, not an ASCII character. `0xC4` indicates a valid stored PIN.

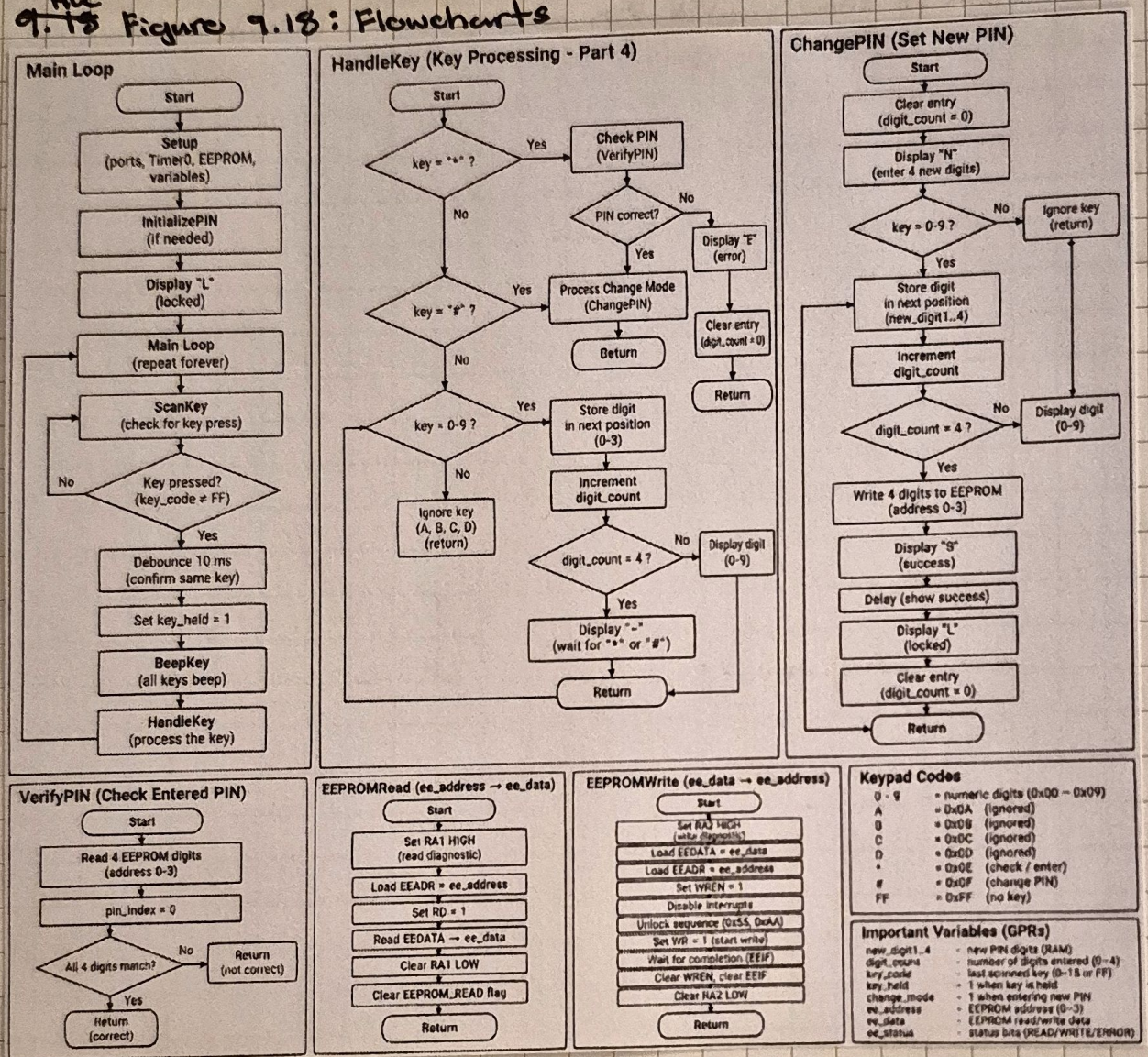
Address	Description
`0x00`	First PIN digit
`0x01`	Second PIN digit
`0x02`	Third PIN digit
`0x03`	Fourth PIN digit
`0x04`	Validity marker (`0xC4` = valid)

In Part 4, the electronic lock program from Part 3 was expanded ~~from~~ HiC to include 3 main operating modes: invalid pin setup, normal locked operation, and Pin change. The program now checks EEPROM when powered on to determine whether a valid pin is stored. If no valid pin is found, the user must create and confirm a new PIN before using the lock. The CurrentPINMatch routine compares the entered PIN with EEPROM, while NewPINMatch checks that the new pin and conformation match. One of the biggest changes was adding the PINWrite routine, which safely updates the four-digit Pin into EEPROM and uses a fifth address as a validity marker. Each EEPROM write is verified, and if an error occurs, the program activates an error LED and stops operation. The keypad scanning, buzzer, and solenoid functions from Part 3 are kept and updated to work with the new modes. Please refer to the next page for the flowchart.

Experiment 9: EEPROM

Before Lab:

9.~~15~~ HLC Figure 9.18: Flowcharts



For the full code, refer to the Github link:
https://github.com/hunterclezie-pixel/Lab09---EEPROM.git

In The Lab:

1.) Bring the keypad and the display, verify that each button press is debounced.
2.) Verify that the lock reads the pin already stored in EEPROM and that the PIN digit do not appear in the source code.
3.) Enter a correct PIN and confirm that the solenoid output activates only after "#". Measure the 1sec High pulse.
4.) Test an incorrect PIN, "#" after fewer than four-digits, a fifth digit, and "*" during entry.
5.) Verify the buzzer responds to all 16 keys.
6.) Power cycle the PIC and confirm that the solenoid output stays inactive until the saved PIN is entered again.

Experiment 9: EEPROM

In The Lab:

Figure 9.19 A: Normal Operation and 1sec Solenoid activation

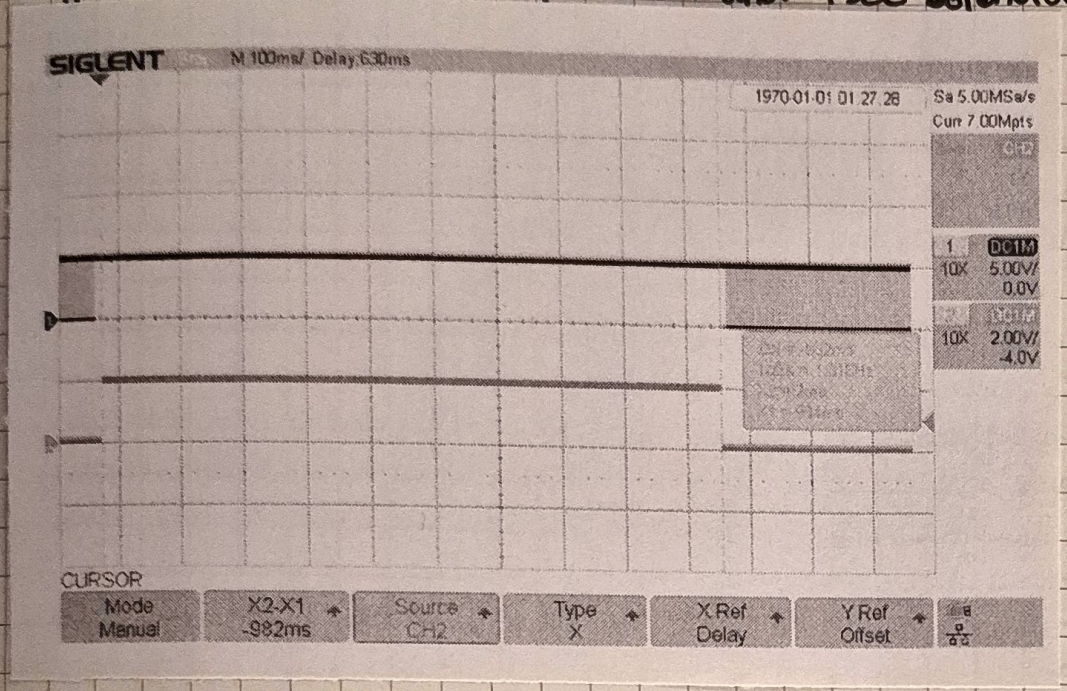
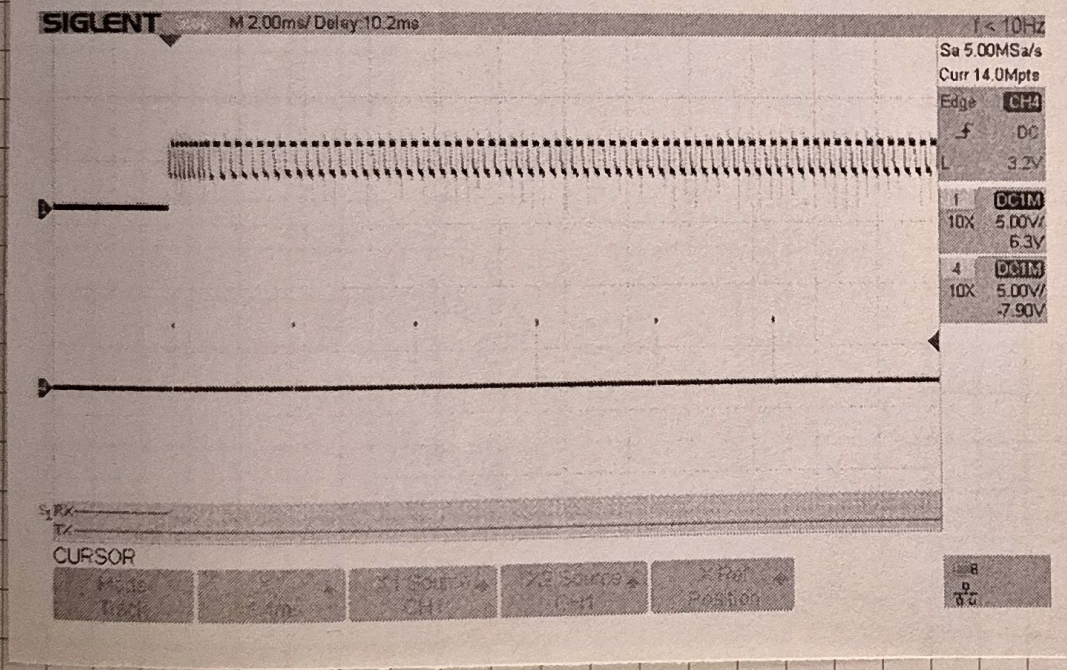


Figure 9.19 B: EEPROM PIN writing Capture



TJR
10-23-24

Experiment 9: EEPROM

Conclusion:

Throughout this lab, the PIC16F883's EEPROM memory was used to store, read, and maintain information even after the microcontroller lost power. In Part 1, EEPROM reading and writing were introduced by using two push buttons ~~even~~ to ~~after~~ MC select and save the characters "A" or "B". Read and write times were measured, and the stored values were tested after power cycling to verify data retention. In Part 2, reusable EEPROM read, write, and verify subroutines were created, allowing the main program to continue operating during writes. Timed selection and errors were developed, allowing the main program to ~~continue~~ MC ~~operating during write~~ HC to prevent unnecessary writes. In Part 3, these routines were used to create a four-digit electronic lock using a keypad, with the entered PIN compared against the stored PIN to activate a simulated solenoid for 1sec. A buzzer also provided audio feedback for each key press. In Part 4, the lock was expanded to allow users to create and change their pins by verifying the current pin, confirming a new one, and safely saving it to EEPROM. Overall, this lab showed how EEPROM, input handling, and verification routines can work together to create a usable and reprogrammable system